

Lecture 3

Treaps

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

1. Introduction
2. Binary Search Trees
3. Self-Balancing Binary Search Trees
4. Treaps

Introduction

- **Problem:** We need a data structure that supports:
 - key search,
 - insertion,
 - deletion.

a.k.a. a set.

Binary Search Trees

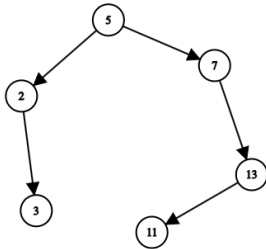
One way to do it: Binary Search Trees

- **Binary Search Tree:**

- each node has at most two children;
- all values in the node's left subtree are smaller than the node's value;
- all values in the node's right subtree are greater than the node's value;
- supports: key searches, insertions, and deletions.

BST Example

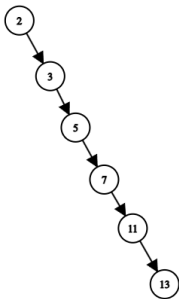
Inserted array: 5, 2, 7, 13, 3, 11



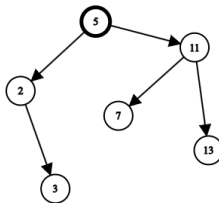
Inorder traversal: 2, 3, 5, 7, 11, 13

More Examples

Inserted array: 2, 3, 5, 7, 11, 13



Inserted array: 5, 2, 11, 3, 7, 13



Self-Balancing Binary Search Trees

Self-balancing Binary Search Trees

- AVL Trees
- Red-Black Trees
- 2-3 Trees
- Splay Trees (**very powerful**)
- Treaps (today's lecture!)

Treaps

What is the Best Insertion Order?

- Assuming we have all the data beforehand, what is the best insertion order?
- Why choose? Just pick a random one!
- We can show that on average, the height will be $O(\log(n))$.

What is a Treap

- In reality we don't have all the data before we start.
- Idea: for each node assign a random insertion order when we first insert it.
- We now need to maintain two key properties:
 - Each node has a lower insertion order than all the nodes in its subtree (heap property).
 - All values in the left subtree are smaller than the node's value and all values in the right subtree are greater (tree property).
- $\text{Tree} + \text{Heap} = \text{Treap!}$

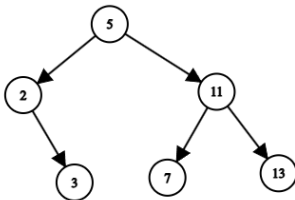
- To facilitate implementing the required set operations, we will define two basic operations:
 - Split
 - Merge

Split Operation

- **Split:** Given a Treap T and a value x , the split operation partitions T into two Treaps:
 - The *left Treap*, denoted T_L , containing all keys $\leq x$.
 - The *right Treap*, denoted T_R , containing all keys $> x$.
- The operation is performed recursively. Let v be the root of the current Treap:
 - If $\text{key}(v) \leq x$, then v belongs to the left Treap. We recursively split the right subtree of v . After the recursive call returns (T_L, T_R) , we set the right child of v to T_L . The final result is (v, T_R) .
 - If $\text{key}(v) > x$, then v belongs to the right Treap. We recursively split the left subtree of v . After the recursive call returns (T_L, T_R) , we set the left child of v to T_R . The final result is (T_L, v) .

Split Example

- Split the given Treap at 10:

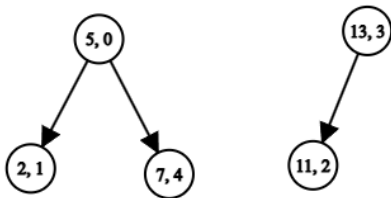


Merge Operation

- **Merge:** Given two Treaps T_L and T_R , such that every key in T_L is strictly less than every key in T_R , the merge operation combines them into a single Treap.
- The operation is performed recursively. Let v_L and v_R be the roots of T_L and T_R , respectively:
 - If $\text{priority}(v_L) < \text{priority}(v_R)$, then v_L becomes the root of the merged Treap. We recursively merge the right subtree of v_L with T_R , and set the result as the new right child of v_L . We then return v_L .
 - Otherwise (i.e., $\text{priority}(v_L) \geq \text{priority}(v_R)$), v_R becomes the root of the merged Treap. We recursively merge T_L with the left subtree of v_R , and set the result as the new left child of v_R . We then return v_R .

Merge Example

- Merge the two given Treaps:



- **Insertion:** To insert a new key x into a Treap T :
 - Split T at x into two Treaps T_L and T_R , where T_L contains all keys $\leq x$ and T_R contains all keys $> x$.
 - Create a new node v with key x and a randomly assigned priority.
 - Merge T_L with v , and then merge the resulting Treap with T_R .

Questions?

- <https://www.cs.umd.edu/class/fall2020/cmsc420-0201/Lects/lect08-treap.pdf>
- https://cp-algorithms.com/data_structures/treap.html
- <https://alexdremov.me/treap-algorithm-explained/>